

Jak działa składanie zamówienia i płatność Stripe

Ten opis dotyczy aktualnego kodu aplikacji Food Ordering Kiosk App. Przepływ zaczyna się w React, przechodzi przez NestJS, warstwę Prisma i kończy na rekordach w PostgreSQL. Najkrótszy obraz procesu: koszyk w bazie danych -> snapshot zamówienia -> Stripe Checkout Session -> redirect do Stripe -> powrót do kiosku -> frontend odświeża status -> backend potwierdza płatność dopiero po zweryfikowanym webhooku Stripe.

1. FRONTEND: WYBÓR PRODUKTÓW I KOSZYK

- Aplikacja startuje w `apps/web/src/app/router.tsx`. Router trzyma stan ekranu: `welcome`, `menu`, `product-details`, `basket-review`, `payment-result`.
- Użytkownik wybiera tryb `dine-in` albo `take-out`. W aktualnym kodzie to jest tylko stan frontendu (`orderMode`), używany w UI. Nie jest wysyłany do API podczas tworzenia zamówienia.
- Gdy użytkownik pierwszy raz dodaje produkt, frontend wywołuje `ensureBasket()`. Jeśli nie ma aktywnego `basketId`, wysyła `POST /api/v1/kiosk/baskets` przez funkcję `createBasket()` z `basketApi.ts`.
- Backend zwraca koszyk z `id`, `sessionId`, listą pozycji i sumą. Frontend zapamiętuje `basket.id` w `basketIdRef`, a pozycje i podsumowanie w stanie React.
- Dodanie produktu idzie przez `addBasketItem()`, czyli `POST /api/v1/kiosk/baskets/:basketId/items`. Ilość zmieniana jest przez `PATCH /api/v1/kiosk/baskets/:basketId/items/:basketItemId`, a usunięcie przez `DELETE /api/v1/kiosk/baskets/:basketId/items/:basketItemId`.

2. BACKEND: TWORZENIE I AKTUALIZACJA KOSZYKA

- Endpointy koszyka są w `apps/api/src/kiosk-basket/kiosk-basket.controller.ts`, a logika w `KioskBasketService`.
- `createBasket()` tworzy rekord `Basket` ze statusem `ACTIVE`. Jeśli request nie ma `sessionId`, backend generuje go przez `randomUUID()`.
- `addItem()` sprawdza, czy koszyk nadal jest aktywny, oraz ładuje produkt menu dostępny dla aktywnej restauracji i menu. Potem `MealConfigurationService` wylicza konfigurację produktu: cenę jednostkową, `fingerprint` i snapshot konfiguracji.
- Zapis pozycji koszyka odbywa się w transakcji Prisma. Jeśli w koszyku istnieje już taka sama pozycja (`basketId` + `menuProductId` + `configurationFingerprint`), backend zwiększa ilość. Jeśli nie, tworzy nowy rekord `BasketItem`.
- Po każdej zmianie backend sumuje `lineTotal` wszystkich pozycji i zapisuje wynik w `Basket.subtotalAmount`. Dzięki temu frontend nie decyduje o cenie końcowej.

3. PRISMA I DB: TABELE KOSZYKA

- `Basket` mapuje się na tabelę `baskets`.
- `BasketItem` mapuje się na `basket_items`.
- `Basket.status` może być m.in. `ACTIVE` albo `CHECKED_OUT`.
- `BasketItem` przechowuje snapshot nazwy produktu, ceny jednostkowej, ilości, `line total` i JSON z konfiguracją. To jest ważne, bo późniejsza zmiana produktu w menu nie powinna zmienić już złożonego koszyka.

4. FRONTEND: KLIKNIECIE "CONTINUE TO PAYMENT"

- Na ekranie `BasketReviewPage` główna funkcja to `handleContinueToPayment()`.
- Funkcja najpierw tworzy snapshot zamówienia: `const order = createdOrder ?? await onCreateOrderSnapshot()`.

- onCreateOrderSnapshot() w routerze wywołuje createOrderSnapshot(basketId), czyli POST /api/v1/kiosk/orders z body { basketId }.
- Po udanym utworzeniu zamówienia router czysci lokalny basketIdRef. Koszyk nie powinien już być dalej edytowany jako aktywny koszyk.
- Dopiero po utworzeniu zamówienia frontend wywołuje createCheckoutSession(order.id), czyli POST /api/v1/kiosk/orders/:orderId/checkout-session.
- Jeśli backend zwróci checkoutUrl, frontend robi window.location.assign(checkoutSession.checkoutUrl). To jest moment, w którym użytkownik opuszcza aplikację i przechodzi do Stripe Checkout.

5. BACKEND: TWORZENIE SNAPSHOTU ZAMOWIENIA

- Endpoint POST /api/v1/kiosk/orders jest w apps/api/src/kiosk-order/kiosk-order.controller.ts.
- KioskOrderService.createOrder() wymaga basketId, a potem w transakcji szuka koszyka BasketStatus.ACTIVE razem z pozycjami, produktami i menu.
- Jeśli koszyk nie istnieje albo nie jest aktywny, backend zwraca błąd BASKET_NOT_ACTIVE. Jeśli jest pusty, zwraca BASKET_EMPTY.
- Backend wylicza restaurantId na podstawie pozycji koszyka. Jeśli pozycje pochodzą z różnych restauracji, zamówienie jest odrzucane.
- Backend liczy subtotalAmount jako sumę BasketItem.lineTotal.
- Tworzony jest rekord Order ze statusem OrderStatus.NEW i paymentStatus = PENDING.
- Razem z zamówieniem tworzone są OrderItem. Każda pozycja dostaje snapshot nazwy produktu, typu, ceny, ilości, line total i konfiguracji.
- Na końcu tej samej transakcji koszyk jest aktualizowany do BasketStatus.CHECKED_OUT. Od tego momentu nie jest już aktywnym koszykiem.

6. PRISMA I DB: TABELE ZAMOWIENIA

- Order mapuje się na orders.
- OrderItem mapuje się na order_items.
- Order.orderNumber ma format podobny do K-YYYYMMDD-XXXXXX, generowany z daty i fragmentu UUID.
- Order.paymentStatus zaczyna jako PENDING, potem przechodzi na AWAITING_PAYMENT_CONFIRMATION, a po webhooku może zostać PAID, FAILED albo CANCELLED.

7. BACKEND: MOMENT ROZPOCZECIA STRIPE

- Stripe startuje dopiero przy POST /api/v1/kiosk/orders/:orderId/checkout-session. To nie jest moment tworzenia koszyka ani nawet samo kliknięcie w UI; faktycznie Stripe jest wywoływany po istnieniu rekordu Order.
- PaymentsService.createCheckoutSession() ładuje zamówienie razem z pozycjami, płatnością i restauracją.
- Backend waliduje, że zamówienie ma pozycje, nie jest już opłacone i ma poprawną kwotę w groszach/centach (toMinorUnits()).
- Tworzona jest sesja Stripe: stripe.checkout.sessions.create({ mode: 'payment', ... }).
- Do Stripe trafia client_reference_id = order.id oraz metadata: orderId i orderNumber. Te dane pomagają powiązać sesję Stripe z zamówieniem.
- success_url i cancel_url prowadzi z powrotem na frontend. URL zawiera checkout=success/cancelled i orderId. Dla sukcesu dodawany jest też session_id={CHECKOUT_SESSION_ID}.
- Kwota Stripe jest jedna pozycja line item: Kiosk order {orderNumber} z kwotą order.totalAmount * 100.
- Po otrzymaniu checkoutSession.url backend zapisuje lub aktualizuje rekord Payment przez payment.upsert().

- W tej samej transakcji backend ustawia `Payment.status` i `Order.paymentStatus` na `AWAITING_PAYMENT_CONFIRMATION`.
- Backend zwraca frontendowi `checkoutUrl`. Frontend przekierowuje użytkownika do Stripe.

8. PRISMA I DB: TABELA PLATNOSCI

- `Payment` mapuje się na `payments`.
- `Payment.orderId` jest unikalne, czyli jedno zamówienie ma maksymalnie jeden rekord płatności.
- `provider` ma wartość `STRIPE`.
- `providerSessionId` przechowuje ID sesji Stripe Checkout i też jest unikalne.
- `providerPaymentIntentId` przechowuje ID `PaymentIntent`, jeśli Stripe je zwrócił.
- `amount` i `currency` są zapisane po stronie backendu na podstawie zamówienia.

9. POWROT ZE STRIPE DO FRONTENDU

- Po Stripe Checkout użytkownik wraca na frontend z parametrami URL, np. `?checkout=success&orderId=...` albo `?checkout=cancelled&orderId=...`.
- `readPaymentReturnState()` w `router.tsx` odczytuje `orderId` i `checkout`. Jeśli jest `orderId`, router pokazuje ekran `payment-result`.
- `PaymentResultPage` od razu odpytuje backend przez `GET /api/v1/kiosk/orders/:orderId`.
- Ekran odświeża status co 2,5 sekundy, dopóki płatność nie ma statusu terminalnego: `PAID`, `FAILED` albo `CANCELLED`.
- Jeżeli Stripe przekierował z sukcesem, ale webhook jeszcze nie dotarł, UI pokazuje stan "Confirming payment". To jest celowe: frontend sam nie potwierdza płatności.

10. BACKEND: WEBHOOK STRIPE, CZYLI PRAWDZIWE POTWIERDZENIE PLATNOSCI

- Webhook odbiera `POST /api/v1/webhooks/stripe` w `apps/api/src/payments/payments.controller.ts`.
- Aplikacja NestJS jest uruchomiona z `rawBody: true` w `main.ts`. To jest potrzebne, bo Stripe wymaga oryginalnego body requestu do weryfikacji podpisu.
- Controller wymaga nagłówka `stripe-signature` oraz `request.rawBody`. Bez tego webhook jest odrzucany.
- `PaymentsService.handleStripeWebhook()` wywołuje `stripe.webhooks.constructEvent(payload, signature, webhookSecret)`. Jeżeli podpis jest niepoprawny, backend zwraca błąd `STRIPE_SIGNATURE_INVALID`.
- `resolvePaymentStatus()` mapuje zdarzenia Stripe: `checkout.session.completed` z `payment_status='paid'` -> `PAID`, `checkout.session.async_payment_succeeded` -> `PAID`, `checkout.session.async_payment_failed` -> `FAILED`, `checkout.session.expired` -> `CANCELLED`.
- `applyCheckoutSessionStatus()` szuka rekordu `Payment` po `providerSessionId = session.id`. Jeśli go nie ma, nic nie aktualizuje.
- Jeśli płatność była już `PAID`, backend nie pozwala nadpisać jej późniejszym statusem innym niż `PAID`.
- W transakcji aktualizowane są dwa miejsca: `Payment.status` oraz `Order.paymentStatus`. To jest moment, w którym zamówienie staje się realnie opłacone.

11. KONFIGURACJA STRIPE I BEZPIECZENSTWO W TYM KODZIE

- `StripeConfigService.getSecretKey()` wymaga `STRIPE_SECRET_KEY` zaczynającego się od `sk_test_`. Kod wymusza Stripe test mode.
- `getWebhookSecret()` wymaga `STRIPE_WEBHOOK_SECRET` zaczynającego się od `whsec_`.
- `FRONTEND_URL` określa, dokąd Stripe ma przekierować użytkownika po sukcesie albo anulowaniu.
- `CORS` w `main.ts` dopuszcza frontend z `FRONTEND_URL` albo domyślnie `http://localhost:5173`.

12. STATUSY: JAK JE CZYTAC

- `Basket.ACTIVE`: koszyk można jeszcze edytować.
- `Basket.CHECKED_OUT`: z koszyka utworzono zamówienie; koszyk nie jest już aktywny.
- `Order.NEW`: zamówienie zostało utworzone, ale w tym kodzie nie ma jeszcze workflow kuchni.
- `PaymentStatus.PENDING`: zamówienie istnieje, ale sesja Stripe jeszcze nie została utworzona.
- `PaymentStatus.AWAITING_PAYMENT_CONFIRMATION`: sesja Stripe istnieje, użytkownik jest lub był w Checkout, backend czeka na webhook.
- `PaymentStatus.PAID`: zweryfikowany webhook potwierdził płatność.
- `PaymentStatus.FAILED`: Stripe zgłosił nieudaną płatność asynchronicznie.
- `PaymentStatus.CANCELLED`: sesja Stripe wygasła albo została uznana za anulowaną.

13. WAŻNE OGRANICZENIA AKTUALNEJ IMPLEMENTACJI

- Tryb `dine-in/take-out` nie jest obecnie zapisywany w DB przy zamówieniu. Jest tylko w stanie `React` i UI.
- `Redirect checkout=success` nie ustawia płatności jako opłaconej. To robi dopiero webhook Stripe.
- W zamówieniu nie widać jeszcze rozbudowanego procesu realizacji kuchennej; `Order.status` startuje jako `NEW`.
- Kwota dla Stripe jest wysyłana jako jedna pozycja "Kiosk order ...", a nie jako osobne produkty z koszyka.

14. NAJWAŻNIEJSZE PLIKI DO CZYTANIA W KOLEJNOŚCI

- `apps/web/src/app/router.tsx` - stan aplikacji, `basketId`, powrót ze Stripe.
- `apps/web/src/pages/BasketReviewPage/BasketReviewPage.tsx` - kliknięcie "Continue to payment".
- `apps/web/src/features/cart/api/basketApi.ts` - requesty HTTP z frontendu.
- `apps/api/src/kiosk-basket/kiosk-basket.service.ts` - tworzenie i przeliczanie koszyka.
- `apps/api/src/kiosk-order/kiosk-order.service.ts` - snapshot zamówienia i checkout koszyka.
- `apps/api/src/payments/payments.service.ts` - Stripe Checkout i webhook.
- `packages/database/prisma/schema.prisma` - modele `Basket`, `Order` i `Payment`.

Dokument wygenerowany na podstawie aktualnego kodu repozytorium, bez zakładania funkcji, które nie są zaimplementowane.